

# Serviço de Notificação de Envio de NFS-e

Documentação Técnica — Painel + Serviço Windows

Versão 1.0 · Março 2026 · Delphi Athens 12.3 VCL

## Colégio Batista Santos Dumont

Sistema NFSe Emissor — Módulo de Agendamento e Notificação

|                     |                                                  |
|---------------------|--------------------------------------------------|
| <b>Projeto:</b>     | prjEmissorNFsPainel + ServicoEnvioNFs            |
| <b>Arquitetura:</b> | MVC — sem IF, sem get/set, interfaces            |
| <b>Comunicação:</b> | Arquivos JSON em C:\ProgramData\ServicoEnvioNFs\ |
| <b>Config:</b>      | NFSe_Servico.ini — secoes [Thread] e [Config]    |

## 1. Visão Geral

Este módulo implementa um mecanismo automatizado de lembrete de envio de NFS-e, composto por dois componentes que se comunicam exclusivamente via arquivos JSON em disco — sem sockets, sem banco de dados, sem dependências externas.

| Componente              | Tipo            | Responsabilidade                                       |
|-------------------------|-----------------|--------------------------------------------------------|
| ServicoEnvioNFs.exe     | Windows Service | Monitora o calendário e grava sinais de notificação    |
| prjEmissorNFsPainel.exe | Aplicação VCL   | Detecta os sinais e interage com o usuário via diálogo |

**Pasta compartilhada:** C:\ProgramData\ServicoEnvioNFs\  
O serviço grava o sinal; o painel lê, exibe ao usuário e grava a resposta.

## 2. Arquivo de Configuração

Todas as configurações são lidas do arquivo **NFSe\_Servico.ini**, localizado na pasta Config\ relativa ao executável. O painel localiza o arquivo subindo até 5 níveis na árvore de diretórios — funciona em qualquer máquina sem ajuste manual.

```
[Thread]
Ativa=0          ; 0 = servico pausado | 1 = servico ativo

[Config]
DiaEnvio=10      ; Dia do mes programado para envio das NFS-e

[ModoEnvio]
EnviarEmLote=1   ; 1 = Lote | 0 = Individual
```

***Importante:** Ativa e EnviarEmLote são gravados como 0 ou 1 (inteiro), não como True/False, garantindo compatibilidade entre o painel e o serviço.*

## 3. Comportamento do Painel

O painel roda na sessão do usuário (com acesso ao desktop) e é o único componente capaz de exibir interfaces visuais ao operador.

**1****Inicialização**

FormCreate sobe a thread TMonitorNotificacao em segundo plano

**2****Monitoramento contínuo**

A thread verifica a pasta compartilhada a cada 30 segundos

**3****Aguarda sinal**

Enquanto notificacao\_pendente.json não existir, dorme e repete

**1 Sinal detectado**

1 Thread encontra notificacao\_pendente.json na pasta compartilhada

**1 Remove o sinal**

2 Apaga o arquivo imediatamente — evita reaparecer no próximo ciclo

**1 Exibe diálogo**

3 "O envio das NFS-e de Março/2026 está programado para o dia 10. Confirma?"

**1 Campo editável**

4 O campo de novo dia vem preenchido com o dia atual e pode ser alterado

## 4. Comportamento do Serviço Windows

O serviço roda na Sessão 0 do Windows, completamente isolado do desktop. Não pode exibir janelas — comunica-se exclusivamente via arquivos.

**5 Serviço instalado e ativo**

Instalação via ServiceEnvioNFs.exe /REINSTALL (requer admin)

**6 Ciclo de verificação**

TWorkerThread acorda a cada 1 hora (30s em modo /DEBUG)

**7 Lê [Thread] Ativa**

Relê o .ini a cada ciclo — alteração tem efeito imediato sem reiniciar

**8 Ativa=0 → pausa**

Ignora o ciclo inteiro, dorme e aguarda o próximo

**9 Ativa=1 → verifica calendário**

Calcula quantos dias faltam para o DiaEnvio configurado

**1 Falta mais de 1 dia → dorme**

0 Nenhuma ação; repete no próximo ciclo

**1 Falta 1 dia → grava sinal**

0 Cria notificacao\_pendente.json em C:\ProgramData\ServiceEnvioNFs\

## 5. Resposta do Usuário e Ação do Serviço

### Caminho SIM — confirma o envio no dia atual

**1 Usuário clica SIM**

5 Painel grava {"confirmou":true,"novo\_dia":0}

- 1** **Serviço lê resposta**  
**6** No próximo ciclo lê resposta\_notificacao.json
- 1** **ConfirmarEnvio**  
**7** Nenhuma alteração no .ini — mantém o DiaEnvio atual
- 1** **Limpeza**  
**8** Apaga resposta\_notificacao.json da pasta compartilhada

### Caminho NÃO — altera o dia de envio

- 1** **Usuário clica NÃO + digita novo dia (ex: 15)**  
**5** Painel grava {"confirmou":false,"novo\_dia":15}
- 1** **Serviço lê resposta**  
**6** No próximo ciclo lê resposta\_notificacao.json
- 1** **AlterarDiaEnvio(15)**  
**7** Grava DiaEnvio=15 na seção [Config] do NFSe\_Servico.ini
- 1** **Limpeza + novo ciclo**  
**8** Apaga resposta\_notificacao.json; próximo ciclo já usa o dia 15

**Validação do novo dia:** o painel aceita apenas valores entre 1 e 31. Fora desse intervalo, mantém automaticamente o dia atual configurado.

## 6. Controle Manual via Tela de Configuração

O operador controla o serviço diretamente pelo painel, sem acessar o Gerenciador de Serviços do Windows.

| Ação na tela                           | Efeito no .ini       | Efeito no serviço                            |
|----------------------------------------|----------------------|----------------------------------------------|
| Thread Desligada (checkbox desmarcado) | Grava Ativa=0        | Para tudo no próximo ciclo — sem reiniciar   |
| Thread Ligada (checkbox marcado)       | Grava Ativa=1        | Retoma no próximo ciclo — sem reiniciar      |
| Alterar Data de Envio (ex: 15)         | Grava DiaEnvio=15    | Próximo ciclo usa o novo dia como referência |
| Selecionar Enviar em Lote              | Grava EnviarEmLote=1 | Define o modo de envio das NFS-e             |
| Selecionar Individual                  | Grava EnviarEmLote=0 | Define o modo de envio das NFS-e             |

## 7. Estrutura dos Arquivos JSON

**notificacao\_pendente.json — gravado pelo serviço:**

```
{
  "dia_envio"      : 10,
  "mes"           : 3,
  "ano"           : 2026,
  "dias_restantes": 1,
  "mensagem"      : "Envio programado para o dia 10"
}
```

#### resposta\_notificacao.json — gravado pelo painel:

```
{ "confirmou": true,  "novo_dia": 0 }    <-- Usuário confirmou
{ "confirmou": false, "novo_dia": 15 }   <-- Usuário alterou para dia 15
```

## 8. Arquivos Implementados

| Arquivo                        | Projeto | Responsabilidade                                       |
|--------------------------------|---------|--------------------------------------------------------|
| Model.IMonitorNotificacao.pas  | Painel  | Interface da thread monitora                           |
| Model.TMonitorNotificacao.pas  | Painel  | Thread que lê JSON e exibe diálogo                     |
| View.Principal.pas             | Painel  | Cria/destrói o monitor no FormCreate/Destroy           |
| Model.IConfiguracaoSistema.pas | Painel  | Record TDadosConfiguracao (+ EnviarEmLote)             |
| Model.TGravarArquivos.pas      | Painel  | GravarIni — WriteInteger para Ativa e EnviarEmLote     |
| Model.TRecuperarArquivos.pas   | Painel  | RecuperarIni — lê [ModoEnvio] EnviarEmLote             |
| Controller.IConfiguracaoView   | Painel  | Assinaturas atualizadas PreencherTela/ColetarTela      |
| Controller.TConfiguracaoView   | Painel  | Implementação completa com novos parâmetros            |
| View.Configuracao.pas          | Painel  | Passa rbEnviarLote/Individual nas chamadas             |
| Service.TWorkerThread.pas      | Serviço | Thread principal — lê Ativa, calcula dias, grava sinal |
| Model.TNotificadorModel.pas    | Serviço | Grava notificacao_pendente.json e lê resposta          |
| Model.TConfiguracaoModel.pas   | Serviço | Lê NFSe_Servico.ini (todas as seções)                  |
| View.TServiceEnvioNFs.pas      | Serviço | Composition Root — sem DFM, sem IF                     |

## 9. Observações Técnicas

- **Sessão 0 vs Sessão do usuário:** o serviço roda na Sessão 0, isolado do desktop. Apenas o painel pode exibir janelas. Por isso a comunicação é via arquivos JSON em C:\ProgramData, pasta acessível por ambos.
- **Thread-safe:** o diálogo é exibido via TThread.Queue(nil, procedure...), garantindo execução na thread principal da VCL.
- **Sem reinicialização:** a chave [Thread] Ativa é relida a cada ciclo. Alterá-la no painel tem efeito imediato sem parar ou reiniciar o serviço.

- **Padrão arquitetural:** MVC estrito — sem IF nas classes de negócio, sem get/set, interfaces para todas as dependências, factory method Criar.
- **FreeOnTerminate := False:** obrigatório na TWorkerThread para que o TService possa chamar WaitFor no OnStop/OnShutdown com segurança.